DOKUMENTASI PEMBELAJARAN

Manajemen Akun User & Route Group dan Middleware

Proyek: Aplikasi TokoOnline (Laravel 10)

Mata Kuliah: Web Programming III

Materi: Minggu ke-4 & Minggu ke-5

Framework: Laravel 10 · PHP 8.1 · MySQL (Laragon)

Referensi Modul: 0688_Modul-Web Programming III

TERMASUK LAMPIRAN MINGGU KE-6

Manajemen Keranjang Belanja

Daftar Isi

Minggu ke-4 — Mengelola Manajemen Akun User

- 4.1 Tujuan Pembelajaran
- 4.2 Konsep: Dua Sisi Manajemen Akun
- 4.3 Frontend — Menampilkan Info Pengguna yang Login
- 4.4 Backend — Halaman Kelola Customer
- 4.5 Perbedaan Modul vs Implementasi
- 4.6 Pengujian

Minggu ke-5 — Route Group dan Middleware

- 5.1 Tujuan Pembelajaran
- 5.2 Konsep: Route Group & Middleware
- 5.3 Membuat Middleware IsCustomer
- 5.4 Registrasi Middleware di Kernel
- 5.5 Controller Akun Customer (akun & updateAkun)
- 5.6 Route Group untuk Customer
- 5.7 View Edit Akun Customer
- 5.8 Perbedaan Modul vs Implementasi
- 5.9 Pengujian

Ringkasan & Alur Lengkap

Minggu ke-6 — Manajemen Keranjang Belanja

- 6.1 Tujuan Pembelajaran
- 6.2 Audit Kondisi Awal
- 6.3 Arsitektur Keranjang yang Dipilih
- 6.4 Struktur Tabel order dan order_item
- 6.5 Model Order dan OrderItem
- 6.6 OrderController
- 6.7 Route dan Middleware
- 6.8 Tampilan Frontend
- 6.9 Perbedaan Modul vs Implementasi
- 6.10 Keamanan dan Konsistensi Data
- 6.11 Pengujian
- 6.12 Daftar File yang Dibuat / Diubah
- 6.13 Alur Pengguna, Batasan, dan Kesimpulan

Minggu ke-4 — Mengelola Manajemen Akun User

4.1 Tujuan Pembelajaran

Setelah materi ini, kita mampu:

- Menampilkan informasi pengguna yang sedang login pada halaman **frontend** menggunakan session autentikasi Laravel.
- Menampilkan menu akun yang berbeda untuk kondisi *sudah login* dan *belum login*.
- Membuat tombol **Logout** yang aman menggunakan form `POST` + token `@csrf`.
- Menampilkan daftar pelanggan (customer) pada halaman **backend** yang datanya terhubung ke tabel `users`.

4.2 Konsep: Dua Sisi Manajemen Akun

Manajemen akun dikelola dari **dua sisi**:

Sisi	Pengguna	Fungsi
Frontend	Pengguna umum (customer)	Mengakses aplikasi, melihat profil sendiri, login & logout.
Backend	Admin / Superadmin	Memantau dan mengelola data pelanggan.

Relasi Tabel `users` dan `customer`

Proyek ini memisahkan identitas login (tabel `users`) dengan data profil pelanggan (tabel `customer`). Keduanya dihubungkan oleh kolom `user_id`.

```
-- Tabel users (kolom penting)
id, nama, email (unique), role (admin|superadmin|customer), status, password, hp, foto

-- Tabel customer
id, user_id  ->  foreign key ke users.id (onDelete cascade),
google_id, google_token, alamat, pos, timestamps
```

Relasi Eloquent pada model: `Customer belongsTo User` dan `User hasOne Customer`.

4.3 Frontend — Menampilkan Info Pengguna yang Login

File: `resources/views/v_layouts/app.blade.php` — pada bagian komentar `<!-- Account -->`.

Kita gunakan `Auth::check()` untuk membedakan kondisi login, dan `Auth::user()` untuk mengambil data pengguna yang benar-benar sedang login (bukan data statis).

```
@if (Auth::check())
<!-- Account -->
<li class="header-account dropdown default-dropdown">
    <div class="dropdown-toggle" role="button" data-toggle="dropdown" aria-expanded="true">
        <div class="header-btns-icon">
            <i class="fa fa-user-o"></i>
        </div>
        <strong class="text-uppercase">{{ Auth::user()->nama ?: 'Akun Saya' }}<i class="fa fa-
caret-down"></i></strong>
    </div>
    <ul class="custom-menu">
        <li><a href="{{ route('customer.akun', ['id' => Auth::user()->id]) }}"><i class="fa fa-
user-o"></i> Akun Saya</a></li>
        <li><a href="#"><i class="fa fa-check"></i> History</a></li>
        <li>
            <a href="#" onclick="event.preventDefault(); document.getElementById('keluar-
app').submit();"><i class="fa fa-power-off"></i> Keluar</a>
            <!-- form keluar app -->
            <form id="keluar-app" action="{{ route('customer.logout') }}" method="POST" class="d-
none">
                @csrf
            </form>
            <!-- form keluar app end -->
        </li>
    </ul>
</li>
@else
<li class="header-account dropdown default-dropdown">
    <div class="dropdown-toggle" role="button" data-toggle="dropdown" aria-expanded="true">
        <div class="header-btns-icon">
            <i class="fa fa-user-o"></i>
        </div>
        <strong class="text-uppercase">Akun Saya<i class="fa fa-caret-down"></i></strong>
    </div>
    <a href="{{ route('auth.redirect') }}" class="text-uppercase">Login</a>
</li>
<!-- /Account -->
@endif
```

Poin penting:

- Saat **belum login** → tampil "Akun Saya" + tombol **Login** menuju `route('auth.redirect')` (login Google dari Minggu 3).
- Saat **sudah login** → tampil `Auth::user()->nama` dan menu dropdown (Akun Saya, History, Keluar).
- Fallback `?: 'Akun Saya'` mencegah tampilan error/kosong bila nama kosong.

4.4 Backend — Halaman Kelola Customer

a. Controller — `app/Http/Controllers/CustomerController.php`

```
public function index()
{
    $customer = Customer::orderBy('id', 'desc')->get();
    return view('backend.v_customer.index', [
        'judul' => 'Customer',
        'sub'    => 'Halaman Customer',
        'index'  => $customer
    ]);
}
```

b. Route — `routes/web.php` (dilindungi middleware `auth`)

```
Route::resource('backend/customer', CustomerController::class, ['as' => 'backend'])
    ->middleware('auth');
```

c. Sidebar backend — `resources/views/backend/v_layouts/app.blade.php` pada komentar `<!-- Sidebar navigation-->`, tambahkan menu Customer ke `route('backend.customer.index')`.

d. View — `resources/views/backend/v_customer/index.blade.php` menampilkan No, Nama, Email, Aksi dengan mengambil data lewat relasi:

```
@foreach ($index as $row)
<tr>
    <td>{{ $loop->iteration }}</td>
    <td>{{ $row->user->nama }}</td>
    <td>{{ $row->user->email }}</td>
    <td>... tombol Detail / Ubah / Hapus ...</td>
</tr>
@endforeach
```

> Kolom **Nama** dan **Email** diambil dari relasi `$row->user` karena data tersebut tersimpan di tabel `users`, bukan di tabel `customer`.

4.5 Perbedaan Modul vs Implementasi

Modul	Implementasi Proyek	Alasan
`{{ Auth::user()->nama }}`	`{{ Auth::user()->nama ?: 'Akun Saya' }}`	Mencegah error bila nama kosong (ketentuan tugas).
Tombol Hapus: `$row->nama`	`$row->user->nama`	Tabel `customer` tidak punya kolom `nama`; cara modul akan error.

4.6 Pengujian

Uji	Harapan	Hasil
Route terdaftar	`customer.akun`, `customer.logout`, `backend.customer.index` ada	Sesuai
Beranda belum login	Tampil "Akun Saya" + tombol Login	Berhasil
Login Google	Nama pengguna tampil di header	Berhasil
Dropdown Akun Saya	Menu Akun Saya / History / Keluar	Berhasil
Logout (form POST + CSRF)	Kembali ke tampilan login	Berhasil
Halaman backend Customer	Data customer tampil dari relasi user	Berhasil

Minggu ke-5 — Route Group dan Middleware

5.1 Tujuan Pembelajaran

- Memahami konsep **Route Group** dan **Middleware** di Laravel.
- Membuat middleware kustom `IsCustomer` untuk membatasi akses hanya bagi pengguna ber-role customer.
- Mengelompokkan route customer dengan `Route::middleware(...)->group()`.
- Membuat halaman **Akun Saya** tempat customer melihat & mengubah datanya sendiri.

5.2 Konsep: Route Group & Middleware

Route Group — mengelompokkan beberapa route dengan konfigurasi sama (prefix, middleware, namespace) agar kode lebih rapi:

```
Route::middleware('is.customer')->group(function () {
    // semua route di sini otomatis memakai middleware is.customer
});
```

Middleware — lapisan yang memproses request *sebelum* mencapai aplikasi. Contoh umum: memastikan hanya user tertentu yang boleh mengakses halaman.

5.3 Membuat Middleware IsCustomer

Buat middleware lewat terminal:

```
php artisan make:middleware IsCustomer
```

File: `app/Http/Middleware/IsCustomer.php`

```
namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class IsCustomer
{
    public function handle(Request $request, Closure $next): Response
    {
        if (auth()->check() && auth()->user()->role == 'customer') {
            return $next($request);
        }

        return redirect('/auth/redirect')
            ->with('msgError', 'Anda harus login sebagai customer');
    }
}
```

> **Perbedaan dengan modul:** modul menulis `auth()->user()->role == 2` (enum numerik versi lama). Pada proyek ini kolom `role` bertipe enum string `admin|superadmin|customer`, sehingga pengecekan disesuaikan menjadi `== 'customer'`. Tanpa penyesuaian ini, customer asli tidak akan pernah lolos.

5.4 Registrasi Middleware di Kernel

File: `app/Http/Kernel.php` — daftarkan alias middleware:

```
protected $middlewareAliases = [
    'auth' => \App\Http\Middleware\Authenticate::class,
    'auth.basic' => \Illuminate\Auth\Middleware\AuthenticateWithBasicAuth::class,
    'is.customer' => \App\Http\Middleware\IsCustomer::class,    // <-- tambahkan
    ...
];
```

> Pada **Laravel 10** properti ini bernama `$middlewareAliases`. Modul menyebut `$routeMiddleware` (nama pada Laravel 8/9).

5.5 Controller Akun Customer

File: `app/Http/Controllers/CustomerController.php`. Tambahkan `use App\Helpers\ImageHelper;` lalu dua method berikut.

Method `akun($id)` — menampilkan halaman akun

```
public function akun($id)
{
    $loggedInCustomerId = Auth::user()->id;
    // Cek apakah ID yang diberikan sama dengan ID customer yang sedang login
    if ($id != $loggedInCustomerId) {
        return redirect()->route('customer.akun', ['id' => $loggedInCustomerId])
            ->with('msgError', 'Anda tidak berhak mengakses akun ini.');
    }
    $customer = Customer::where('user_id', $id)->firstOrFail();
    return view('v_customer.edit', [
        'judul'    => 'Akun Saya',
        'subJudul' => 'Akun Customer',
        'edit'     => $customer
    ]);
}
```

Method `updateAkun(Request $request, $id)` — menyimpan perubahan

```
public function updateAkun(Request $request, $id)
{
    $customer = Customer::where('user_id', $id)->firstOrFail();

    // Pastikan hanya pemilik akun yang sedang login yang boleh mengubah
    if (Auth::user()->id != $id) {
        return redirect()->route('customer.akun', ['id' => Auth::user()->id])
            ->with('msgError', 'Anda tidak berhak mengubah akun ini.');
    }

    $rules = [
        'nama' => 'required|max:255',
        'hp'   => 'required|min:10|max:13',
        'foto' => 'image|mimes:jpeg,jpg,png,gif|file|max:1024',
    ];
    $messages = [
        'foto.image' => 'Format gambar gunakan file dengan ekstensi jpeg, jpg, png, atau gif.',
        'foto.max'   => 'Ukuran file gambar Maksimal adalah 1024 KB.'
    ];

    if ($request->email != $customer->user->email) {
        $rules['email'] = 'required|max:255|email|unique:users';
    }
    if ($request->alamat != $customer->alamat) {
        $rules['alamat'] = 'required';
    }
    if ($request->pos != $customer->pos) {
        $rules['pos'] = 'required';
    }

    $validatedData = $request->validate($rules, $messages);

    // Upload & resize foto bila ada file baru
    if ($request->file('foto')) {
        if ($customer->user->foto) {
            $oldImagePath = public_path('storage/img-customer/') . $customer->user->foto;
            if (file_exists($oldImagePath)) {
                unlink($oldImagePath);
            }
        }
        $file = $request->file('foto');
        $extension = $file->getClientOriginalExtension();
        $originalFileName = date('YmdHis') . '_' . uniqid() . '.' . $extension;
        $directory = 'storage/img-customer/';
        ImageHelper::uploadAndResize($file, $directory, $originalFileName, 385, 400);
        $validatedData['foto'] = $originalFileName;
    }

    $customer->user->update($validatedData);

    $customer->update([
        'alamat' => $request->input('alamat'),
        'pos'    => $request->input('pos'),
    ]);

    return redirect()->route('customer.akun', $id)
```

```
        ->with('success', 'Data berhasil diperbarui');
}
```

Perbedaan modul vs proyek:

- Modul menulis `unique:customer`, padahal kolom `email` berada di tabel `users` → diganti `unique:users` agar validasi benar-benar berjalan.
- Ditambah pengecekan kepemilikan akun (`Auth::user()->id != $id`) di `updateAkun()` sebagai pengaman tambahan.

Jangan lupa membuat folder penyimpanan: `storage/app/public/img-customer/`

5.6 Route Group untuk Customer

File: `routes/web.php`

```
// Group route untuk customer (Minggu 5: Route Group & Middleware)
Route::middleware('is.customer')->group(function () {
    // Halaman akun customer
    Route::get('/customer/akun/{id}', [CustomerController::class, 'akun'])
        ->name('customer.akun');

    // Update data akun customer
    Route::put('/customer/updateakun/{id}', [CustomerController::class, 'updateAkun'])
        ->name('customer.updateakun');
});
```

Menyambungkan menu "Akun Saya"

File: `resources/views/v_layouts/app.blade.php` (bagian `<!-- Account -->`):

```
<li>
    <a href="{{ route('customer.akun', ['id' => Auth::user()->id]) }}">
        <i class="fa fa-user-o"></i> Akun Saya
    </a>
</li>
```

5.7 View Edit Akun Customer

File baru: `resources/views/v_customer/edit.blade.php` (extends `v_layouts.app`).

```
@extends('v_layouts.app')
@section('content')
<div class="row">
  <div class="col-md-12">
    <div class="billing-details">
      <div class="section-title"><h3 class="title">{{ $judul }}</h3></div>

      @if(session()->has('success'))
        <div class="alert alert-success">{{ session('success') }}</div>
      @endif
      @if(session()->has('msgError'))
        <div class="alert alert-danger">{{ session('msgError') }}</div>
      @endif

      <form action="{{ route('customer.updateakun', $edit->user->id) }}"
            method="post" enctype="multipart/form-data">
        @method('put')
        @csrf

        <!-- Foto -->
        @if ($edit->user->foto)
          <img src="{{ asset('storage/img-customer/' . $edit->user->foto) }}" width="100%">
        @else
          <img src="{{ asset('storage/img-user/img-default.jpg') }}" width="100%">
        @endif
        <input type="file" name="foto">

        <!-- Nama, Email, HP, Alamat, Kode Pos ... -->
        <input type="text" name="nama" value="{{ old('nama', $edit->user->nama) }}">
        <input type="text" name="email" value="{{ old('email', $edit->user->email) }}">
        <input type="text" name="hp" value="{{ old('hp', $edit->user->hp) }}">
        <textarea name="alamat">{{ old('alamat', $edit->alamat) }}</textarea>
        <input type="text" name="pos" value="{{ old('pos', $edit->pos) }}">

        <button type="submit" class="primary-btn">Simpan</button>
      </form>
    </div>
  </div>
</div>
@endsection
```

Perbedaan modul: modul menulis `@if ($edit->foto)`, padahal tabel `customer` tidak memiliki kolom `foto` (foto tersimpan di tabel `users`) → diganti `@if ($edit->user->foto)`. Ditambahkan juga fungsi JS kecil `hanyaAngka()` dan `previewFoto()` karena tidak tersedia di `main.js` template.

5.8 Perbedaan Modul vs Implementasi

Modul	Implementasi Proyek	Alasan
`role == 2`	`role == 'customer'`	Kolom role proyek berupa enum string.
`$routeMiddleware`	`$middlewareAliases`	Nama properti pada Laravel 10.
`unique:customer`	`unique:users`	Kolom email ada di tabel users.
`@if ($edit->foto)`	`@if ($edit->user->foto)`	Tabel customer tak punya kolom foto.
—	Cek kepemilikan di `updateAkun()`	Pengaman tambahan.
—	Relasi `User::customer()`	Melengkapi pasangan relasi `Customer belongsTo User`.

5.9 Pengujian

Skenario	Harapan	Hasil
Guest buka `/customer/akun/1`	Redirect ke `/auth/redirect` (Google)	Berhasil
Admin buka `/customer/akun/{id}`	Diblokir middleware (role bukan customer)	Berhasil
Customer buka akun sendiri	Halaman "Akun Saya" tampil	Berhasil
Customer buka akun orang lain	Ditolak + `msgError`	Berhasil
Update akun (nama, alamat)	Data tersimpan	Berhasil
Edit email ke email milik user lain	Gagal validasi `unique:users`	Berhasil
Link "Akun Saya" di beranda	Mengarah ke `customer.akun`	Berhasil

> **Total pengujian otomatis:** 7 skenario, 17 assertion — **semua lulus**. Halaman beranda & produk tetap berjalan normal (HTTP 200).

Ringkasan & Alur Lengkap

Daftar File yang Dibuat / Diubah

File	Status	Keterangan
`resources/views/v_layouts/app.blade.php`	Diubah	Menu akun frontend (Minggu 4 & 5).
`app/Http/Controllers/CustomerController.php`	Diubah	`index`, `akun`, `updateAkun`, `logout`.
`resources/views/backend/v_customer/index.blade.php`	Ada	Halaman kelola customer (Minggu 4).
`app/Http/Middleware/IsCustomer.php`	Dibuat	Middleware pembatas role customer.
`app/Http/Kernel.php`	Diubah	Registrasi alias `is.customer`.
`routes/web.php`	Diubah	Route Group customer.
`app/Models/User.php`	Diubah	Relasi `customer()`.
`resources/views/v_customer/edit.blade.php`	Dibuat	Form Edit Akun customer.
`storage/app/public/img-customer/`	Dibuat	Folder foto customer.

Alur Pengguna (Customer)

1. Buka beranda → klik **Login** → diarahkan ke Google (`auth.redirect`).
2. Setelah login Google, sistem membuat/menyambungkan akun di tabel `users` + `customer` (tanpa duplikat).
3. Nama pengguna muncul di header; klik **Akun Saya** → middleware `is.customer` memverifikasi role.
4. Customer mengubah data (nama, email, hp, alamat, pos, foto) → disimpan ke `users` & `customer`.
5. Klik **Keluar** → form POST + CSRF ke `customer.logout` → kembali ke tampilan login.

Catatan Pembelajaran Penting

- **Keamanan logout:** gunakan `POST` + `@csrf`, bukan link `GET`, untuk mencegah CSRF.
- **Ambil data asli:** gunakan `Auth::user()` dari session, jangan data statis.
- **Middleware** memusatkan aturan akses sehingga tidak perlu menulis pengecekan di setiap controller.
- **Route Group** membuat konfigurasi middleware diterapkan sekaligus ke beberapa route.
- Selalu sesuaikan contoh modul dengan kondisi proyek (nama kolom, tipe data, versi Laravel).

T

DOKUMENTASI PEMBELAJARAN

Manajemen Keranjang Belanja

Proyek: Aplikasi TokoOnline (Laravel 10)

Mata Kuliah: Web Programming III

Materi: Minggu ke-6

Framework: Laravel 10 · PHP 8.1 · MySQL (Laragon)

Referensi Modul: 0688_Modul-Web Programming III (hlm. 59–65)

STATUS: SELESAI & TERUJI

Minggu ke-6 — Manajemen Keranjang Belanja

6.1 Tujuan Pembelajaran

Setelah materi ini, kita mampu:

- Menambahkan produk ke keranjang berdasarkan ID produk yang valid.
- Menampilkan gambar, nama, harga, jumlah, subtotal, total berat, dan total harga.
- Mengubah jumlah produk dan menghapus produk dari keranjang.
- Menyimpan keranjang secara permanen menggunakan tabel **order** dan **order_item**.
- Menampilkan pesan sukses/error, kondisi keranjang kosong, dan badge jumlah item pada navbar.
- Menjaga integrasi autentikasi Laravel dan login Google yang sudah ada.

> **Hasil akhir:**
> Fitur keranjang Pertemuan 6 berfungsi pada proyek TokoOnline yang sudah ada, mengikuti modul UBSI, tanpa menjalankan **migrate:fresh** dan tanpa merusak fitur produk, kategori, akun, maupun login Google.

6.2 Audit Kondisi Awal

Audit dilakukan sebelum menulis kode agar implementasi cocok dengan struktur proyek yang benar-benar ada. Proyek berada di `C:\Laravel10\TokoOnline`.

Bagian yang diaudit	Temuan	Dampak pada implementasi
Route	Belum ada route cart/order.	Empat route baru ditambahkan.
Controller & model	OrderController, Order, dan OrderItem belum ada.	Dibuat sesuai kebutuhan modul.
View keranjang	v_order/cart.blade.php belum ada.	Dibuat dengan template TokoOnline.
Tabel database	order dan order_item belum ada.	Migrasi baru dijalankan tanpa menghapus tabel lama.
Migrasi order	File create_order_table ada, tetapi hanya berisi id + timestamps dan belum tercatat di tabel migrations.	Stub dilengkapi, lalu dimigrasikan.
Autentikasi	Login memakai tabel users; profil customer di tabel customer dengan FK customer.user_id ke users.id.	Keranjang memakai users.id dan customer.id, bukan tabel user.
Role pengguna	users.role bertipe enum string admin|superadmin|customer.	Middleware is.customer dipakai apa adanya.
Produk	produk memiliki harga, stok, berat, status aktif/blok, dan foto.	Harga, stok, berat, dan status diambil dari database.
Tombol Pesan	Form masih memakai action="3" atau "#".	Semua diarahkan ke route order.addToCart.
MySQL Laragon	Service sempat berhenti.	Dinyalakan untuk audit, migrasi, dan pengujian.

6.3 Arsitektur Keranjang yang Dipilih

Modul menggunakan pendekatan **database**: order berstatus **pending** berfungsi sebagai keranjang milik customer. Pendekatan ini dipilih agar data keranjang tetap ada setelah pengguna keluar-masuk halaman, dan agar tidak ada sistem ganda antara session cart dan database cart.

Alur request

```
Customer klik tombol "Pesan"
  -> middleware is.customer (harus login sebagai customer)
  -> OrderController@addToCart
  -> validasi produk, status, dan stok
  -> DB transaction: cari/buat order pending + simpan order_item
  -> recalculate total_harga dan total_berat dari seluruh item
  -> redirect ke order.cart
  -> v_order/cart.blade.php menampilkan item dan total
```

Relasi utama

- **Order** belongsTo **Customer** dan belongsTo **User**; hasMany **OrderItem**.
- **OrderItem** belongsTo **Order** dan belongsTo **Produk**.
- Satu baris **order_item** mewakili satu produk dalam keranjang; produk yang sama tidak dibuat baris baru, tetapi quantity-nya ditambah.
- Constraint unik **(order_id, produk_id)** menjaga agar tidak ada duplikasi baris.

> **Session, database, dan AJAX.**
> Session tetap dipakai untuk autentikasi dan flash message sukses/error. Data keranjang disimpan di database. AJAX tidak ditambahkan karena modul menggunakan form POST biasa; pendekatan ini lebih sederhana, konsisten, dan tetap memenuhi seluruh kebutuhan Pertemuan 6 tanpa membuat sistem ganda.

6.4 Struktur Tabel order dan order_item

Struktur mengikuti modul, dengan penyesuaian pada foreign key: tabel **users** (bukan **user**) dan tabel **customer**. Kolom **user_id** ditambahkan ke model agar kepemilikan keranjang dapat diperiksa langsung dari user yang sedang login.

Tabel	Kolom penting	Keterangan
`order`	id, customer_id, user_id, status, noresi, kurir, layanan_ongkir, biaya_ongkir, estimasi_ongkir, total_berat, total_harga, alamat, pos, timestamps	customer_id FK ke customer.id; user_id FK ke users.id; status default pending.
`order_item`	id, order_id, produk_id, quantity, harga, timestamps	order_id FK ke order.id; produk_id FK ke produk.id; unik (order_id, produk_id).

database/migrations/2026_10_24_024430_create_order_table.php

```
Schema::create('order', function (Blueprint $table) {
    $table->id();
    $table->unsignedBigInteger('customer_id');
    $table->unsignedBigInteger('user_id')->nullable();
```

database/migrations/2026_10_24_024430_create_order_table.php

```
    $table->string('status')->default('pending');
    $table->string('noresi')->nullable();
    $table->string('kurir')->nullable();
    $table->string('layanan_ongkir')->nullable();
    $table->string('biaya_ongkir')->nullable();
    $table->string('estimasi_ongkir')->nullable();
    $table->integer('total_berat')->nullable();
    $table->double('total_harga')->default(0);
    $table->text('alamat')->nullable();
    $table->string('pos')->nullable();
    $table->timestamps();
    $table->foreign('customer_id')->references('id')->on('customer')
        ->onDelete('cascade');
    $table->foreign('user_id')->references('id')->on('users')
        ->onDelete('cascade');
});
```

database/migrations/2026_10_24_024431_create_order_item_table.php

```
Schema::create('order_item', function (Blueprint $table) {
    $table->id();
    $table->unsignedBigInteger('order_id');
    $table->unsignedBigInteger('produk_id');
    $table->integer('quantity');
    $table->double('harga');
    $table->timestamps();
    $table->unique(['order_id', 'produk_id']);
    $table->foreign('order_id')->references('id')->on('order')
        ->onDelete('cascade');
    $table->foreign('produk_id')->references('id')->on('produk')
        ->onDelete('cascade');
});
```

6.5 Model Order dan OrderItem

Model memakai nama tabel singular sesuai modul: **order** dan **order_item**. Relasi inilah yang dipakai controller dan view untuk mengambil item beserta data produknya.

app/Models/Order.php

```
class Order extends Model
{
    protected $table = 'order';
    protected $fillable = [
        'customer_id', 'user_id', 'status', 'noresi', 'kurir',
        'layanan_ongkir', 'biaya_ongkir', 'estimasi_ongkir',
        'total_berat', 'total_harga', 'alamat', 'pos',
    ];
    public function orderItems()
    {
        return $this->hasMany(OrderItem::class);
    }
    public function customer()
    {
        return $this->belongsTo(Customer::class);
```

app/Models/Order.php

```
    }
    public function user()
    {
        return $this->belongsTo(User::class);
    }
}
```

app/Models/OrderItem.php

```
class OrderItem extends Model
{
    protected $table = 'order_item';
    protected $fillable = ['order_id', 'produk_id', 'quantity', 'harga'];
    public function order()
    {
        return $this->belongsTo(Order::class);
    }
    public function produk()
    {
        return $this->belongsTo(Produk::class);
    }
}
```

Catatan penting.
Kolom **user_id** dimasukkan ke **$fillable** meskipun tidak ditulis di contoh modul. Tanpa kolom ini, keranjang tetap bisa dibuat, tetapi kepemilikan user tidak dapat diperiksa secara langsung saat update/delete.

6.6 OrderController

Empat method utama menangani seluruh siklus keranjang: tambah, tampilkan, ubah jumlah, dan hapus. Semua operasi yang mengubah data dibungkus **DB::transaction**.

Method	Tugas
`addToCart($id)`	Validasi produk dan stok, cari/buat order pending, tambah quantity atau buat item baru, hitung ulang total.
`viewCart()`	Memuat order pending beserta item dan produk, menghitung total harga/berat/quantity, lalu menampilkan v_order.cart.
`updateCart(Request, $id)`	Validasi quantity minimal 1, cek kepemilikan item, cek stok, perbarui quantity dan harga dari database, hitung ulang total.
`removeFromCart($id)`	Cek kepemilikan item, hapus item, hitung ulang total, hapus order jika keranjang sudah kosong.

Inti addToCart()

```
$customer = Customer::where('user_id', Auth::id())->first();
$produk = Produk::find($id);
DB::transaction(function () use ($customer, $produk) {
    $order = Order::firstOrCreate(
        [
```

Inti addToCart()

```
            'customer_id' => $customer->id,
            'user_id' => Auth::id(),
            'status' => 'pending',
        ],
        ['total_harga' => 0, 'total_berat' => 0]
    );
    $item = OrderItem::where('order_id', $order->id)
        ->where('produk_id', $produk->id)
        ->lockForUpdate()
        ->first();
    $quantity = $item ? $item->quantity + 1 : 1;
    if ($quantity > $produk->stok) {
        throw new RuntimeException('Jumlah produk melebihi stok.');
    }
    if ($item) {
        $item->update(['quantity' => $quantity, 'harga' => $produk->harga]);
    } else {
        OrderItem::create([
            'order_id' => $order->id,
            'produk_id' => $produk->id,
            'quantity' => 1,
            'harga' => $produk->harga,
        ]);
    }
    $this->recalculateOrder($order);
});
```

Perhitungan total di server

```
private function recalculateOrder(Order $order): void
{
    $totalHarga = 0;
    $totalBerat = 0;
    foreach ($order->orderItems()->with('produk')->get() as $item) {
        $totalHarga += $item->harga * $item->quantity;
        $totalBerat += ($item->produk->berat ?? 0) * $item->quantity;
    }
    $order->update([
        'total_harga' => $totalHarga,
        'total_berat' => $totalBerat,
    ]);
}
```

Update dan hapus tetap aman.

Item tidak pernah dicari hanya dari ID. Pencarian selalu dibatasi **order_id** milik customer/user yang login, lalu diulang dengan **lockForUpdate()** di dalam transaksi. Jika item bukan miliknya, user hanya mendapat pesan error dan data tidak berubah.

6.7 Route dan Middleware

Semua route keranjang berada dalam group **is.customer**. Guest yang menekan tombol Pesan diarahkan ke login Google, sedangkan admin/superadmin tidak dapat mengakses keranjang customer.

Method	URI	Nama route	Controller
POST	`/add-to-cart/{id}`	`order.addToCart`	addToCart
GET	`/cart`	`order.cart`	viewCart
PUT	`/cart/update/{id}`	`order.updateCart`	updateCart
DELETE	`/cart/remove/{id}`	`order.removeFromCart`	removeFromCart

routes/web.php (di dalam group is.customer)

```
Route::middleware('is.customer')->group(function () {
    Route::get('/cart', [OrderController::class, 'viewCart'])
        ->name('order.cart');
    Route::post('/add-to-cart/{id}', [OrderController::class, 'addToCart'])
        ->whereNumber('id')
        ->name('order.addToCart');
    Route::put('/cart/update/{id}', [OrderController::class, 'updateCart'])
        ->whereNumber('id')
        ->name('order.updateCart');
    Route::delete('/cart/remove/{id}', [OrderController::class, 'removeFromCart'])
        ->whereNumber('id')
        ->name('order.removeFromCart');
});
```

whereNumber('id') mencegah ID non-numerik masuk ke controller. Nama route memakai prefix **order.** agar konsisten dengan modul.

6.8 Tampilan Frontend

Halaman keranjang memakai template TokoOnline yang sudah ada (**v_layouts.app**) dan class Bootstrap seperti **shopping-cart-table**, **table-responsive**, dan **primary-btn**.

- Tabel menampilkan gambar, nama produk, berat, stok, harga, quantity, subtotal, dan aksi hapus.
- Gambar memakai **thumb_sm_** dengan fallback **no-image.jpg**.
- Jumlah dapat diubah lewat form PUT dan dihapus lewat form DELETE + konfirmasi.
- Total berat, total harga, dan jumlah item dihitung dari seluruh item.
- Jika kosong, tampil pesan **Keranjang belanja kosong.** beserta tombol Lihat Produk.
- Pesan sukses/error tampil sebagai alert Bootstrap.

Badge jumlah item pada navbar (v_layouts/app.blade.php)

```
@php
$cartCount = 0;
if (Auth::check() && Auth::user()->role === 'customer') {
    $cartCount = (int) \App\Models\OrderItem::whereHas('order', function ($q) {
        $q->where('user_id', Auth::id())->where('status', 'pending');
    })->sum('quantity');
```

Badge jumlah item pada navbar (v_layouts/app.blade.php)

```
}
@endphp
<a href="{{ route('order.cart') }}">
    <i class="fa fa-shopping-cart"></i>
    @if ($cartCount > 0)
        <span class="qty">{{ $cartCount }}</span>
    @endif
    <strong class="text-uppercase">Keranjang</strong>
</a>
```

Tombol Pesan pada halaman produk

```
<form action="{{ route('order.addToCart', $row->id) }}" method="post"
    style="display: inline-block;" title="Pesan Ke Aplikasi">
    @csrf
    <button type="submit" class="primary-btn add-to-cart">
        <i class="fa fa-shopping-cart"></i> Pesan
    </button>
</form>
```

6.9 Perbedaan Modul vs Implementasi

Modul	Implementasi proyek	Alasan
`FK user_id ke tabel user`	`FK ke tabel users`	Tabel login proyek bernama users.
`create_order hanya id + timestamps`	`Kolom order lengkap sesuai blueprint`	Stub yang belum dimigrasikan dilengkapi.
`order_item tanpa constraint unik`	`unique(order_id, produk_id)`	Mencegah baris produk ganda.
`where('status','pending','paid')`	`where('status','pending')`	Parameter ketiga bukan nilai OR; status keranjang hanya pending.
`Total order ditambah manual`	`Total dihitung ulang dari seluruh item`	Mencegah total drift saat update/hapus.
`Hanya add dan view`	`Ditambah update dan delete`	Kebutuhan manajemen keranjang.
`Tanpa cek kepemilikan`	`Item dibatasi order milik user/customer`	Mencegah perubahan keranjang orang lain.
`Tanpa validasi stok`	`Cek status aktif dan stok`	Mencegah pesanan melebihi stok.
`Form action # atau 3`	`Route order.addToCart`	Tombol Pesan benar-benar berfungsi.
`Tombol Pilih Pengiriman`	`Tombol Lanjutkan Belanja`	Checkout/ongkir bukan cakupan Pertemuan 6.

6.10 Keamanan dan Konsistensi Data

- **Harga dari database.** Input harga dari browser tidak pernah dipakai; harga diambil dari **produk.harga**.

- **Quantity tervalidasi.** Wajib integer dan minimal 1; jumlah di atas stok ditolak.
- **Produk tervalidasi.** Produk harus ada, berstatus aktif, dan stoknya lebih dari 0.
- **Kepemilikan item.** Update/delete selalu mencari item melalui order milik user yang sedang login, bukan hanya dari ID item.
- **Transaksi database.** Perubahan order dan item dibungkus transaksi; baris dikunci dengan **lockForUpdate()**.
- **CSRF dan method.** Semua form memakai **@csrf**; update memakai **@method('put')**, hapus memakai **@method('delete')**.
- **Total di server.** Subtotal, total harga, dan total berat dihitung di server.
- **Data lama aman.** Tidak ada **migrate:fresh**, drop tabel, atau penghapusan data.
- **Autentikasi tetap.** Login Laravel dan Google tidak diubah; secret tetap di .env.

6.11 Pengujian

Pengujian dilakukan dengan Laravel feature test yang benar-benar mengirim request HTTP ke route keranjang. Setiap test dibungkus **DatabaseTransactions**, sehingga data asli tidak berubah.

Perintah yang dijalankan

```
php artisan migrate --pretend
php artisan migrate --force
php artisan route:list --name=order
php artisan view:cache
php artisan view:clear
php vendor\bin\pint --test app\Http\Controllers\OrderController.php ...
php artisan test --filter=OrderCartTest
php artisan test
```

Skenario	Harapan	Hasil
Guest klik Pesan	Dialihkan ke login Google, tidak ada item dibuat	Lulus
Tambah satu produk	Order pending + item quantity 1, redirect ke cart	Lulus
Tambah produk yang sama	Quantity menjadi 2, total Rp. 30.000, berat 500 g	Lulus
Ubah quantity menjadi 3	Total Rp. 36.000; quantity 0 ditolak validasi	Lulus
Ubah melebihi stok	Error, quantity lama tidak berubah	Lulus
Hapus produk	Item hilang, order kosong dihapus, tampil pesan kosong	Lulus
Buka cart tanpa order	Tampil 'Keranjang belanja kosong.'	Lulus
Isolasi customer A/B	B tidak bisa lihat/ubah/hapus item A; order terpisah	Lulus
Produk blok / stok 0	Ditolak, pesan error tampil di halaman cart	Lulus
Badge navbar	Menampilkan total quantity keranjang	Lulus
Fitur lama	5 test Google OAuth + halaman produk tetap berjalan	Lulus

Hasil akhir pengujian.

17 test lulus, 98 assertion. Sebanyak 10 test khusus keranjang dan 7 test lama (unit, redirect beranda, serta Google OAuth). Setelah pengujian, jumlah baris **order** dan **order_item** di database asli tetap 0 karena transaksi test di-rollback.

6.12 Daftar File yang Dibuat / Diubah

File	Status	Keterangan
`app/Models/Order.php`	Dibuat	Model order, relasi item/customer/user.
`app/Models/OrderItem.php`	Dibuat	Model item, relasi order/produk.
`app/Http/Controllers/OrderController.php`	Dibuat	addToCart, viewCart, updateCart, removeFromCart, recalculate.
`resources/views/v_order/cart.blade.php`	Dibuat	Halaman keranjang, update, hapus, total, empty state.
`database/migrations/2026_10_24_024430_create_order_table.php`	Diubah	Stub dilengkapi seluruh kolom order.
`database/migrations/2026_10_24_024431_create_order_item_table.php`	Dibuat	Tabel order_item + foreign key + unique.
`routes/web.php`	Diubah	Empat route keranjang di group is.customer.
`resources/views/v_layouts/app.blade.php`	Diubah	Link cart dan badge jumlah item.
`resources/views/v_beranda/index.blade.php`	Diubah	Action tombol Pesan.
`resources/views/v_produk/index.blade.php`	Diubah	Action tombol Pesan.
`resources/views/v_produk/detail.blade.php`	Diubah	Action tombol Pesan.
`resources/views/v_produk/produkkategori.blade.php`	Diubah	Action tombol Pesan.
`tests/Feature/OrderCartTest.php`	Dibuat	10 test keranjang.
`tests/Feature/ExampleTest.php`	Diubah	Assertion lama disesuaikan dengan redirect / ke beranda.

6.13 Alur Pengguna, Batasan, dan Kesimpulan

Alur pengguna

1. Customer login melalui Google.
2. Customer menekan tombol **Pesan** pada beranda, daftar produk, kategori, atau detail.
3. Middleware **is.customer** memastikan pengguna ber-role customer.
4. Controller memvalidasi produk dan stok, lalu menyimpan order pending + item.
5. Customer diarahkan ke halaman keranjang dan melihat gambar, harga, quantity, subtotal, total.

6. Customer mengubah quantity atau menghapus item; total langsung dihitung ulang di server.

7. Badge pada ikon keranjang mengikuti total quantity order pending.

Batasan yang disengaja

- Checkout, ongkos kirim/RajaOngkir, dan payment gateway belum dikerjakan (materi berikutnya).
- Keranjang hanya untuk user ber-role customer, sesuai middleware dan alur login Google.
- Stok diperiksa saat tambah/update, tetapi belum ada reservasi stok permanen.
- Halaman History masih placeholder; belum ada daftar riwayat order.
- Tidak ada AJAX karena modul memakai form POST; fungsionalitas tetap lengkap.

> **Kesimpulan.**
> Manajemen Keranjang Belanja Pertemuan 6 sudah berfungsi, mengikuti modul UBSI, menggunakan tabel order/order_item yang benar, aman terhadap perubahan data pengguna lain, dan tidak merusak fitur sebelumnya.

Lampiran — Bukti Verifikasi

Ringkasan bukti dari terminal. Output panjang dipangkas agar mudah dibaca.

Status migrasi

```
2026_10_24_024430_create_order_table ......................... [7] Ran
2026_10_24_024431_create_order_item_table .................... [7] Ran
```

Route keranjang

```
POST    add-to-cart/{id}        order.addToCart       OrderController@addToCart
GET     cart                    order.cart            OrderController@viewCart
PUT     cart/update/{id}        order.updateCart      OrderController@updateCart
DELETE  cart/remove/{id}        order.removeFromCart  OrderController@removeFromCart
```

Hasil test

```
PASS  Tests\Feature\OrderCartTest
  guest is redirected to google login when adding to cart
  customer can add same product twice and total is calculated
  customer can update quantity and invalid quantity is rejected
  customer cannot update quantity beyond stock
  customer can remove product and empty cart is displayed
  cart page shows empty state without order
  cart data is isolated between customers
  inactive product and out of stock product are rejected
  navbar shows total cart quantity
  existing frontend product pages still render
Tests: 17 passed (98 assertions)
```

Pemeriksaan gaya kode

```
php vendor\bin\pint --test ...  -> PASS (6 files)
php artisan view:cache           -> Blade templates cached successfully
```

Catatan.
MySQL Laragon sempat berhenti dan dinyalakan untuk audit, migrasi, dan pengujian. File PDF asli Minggu 4–5 tetap dipertahankan; bagian Minggu ke-6 dilampirkan setelah halaman terakhir.